



MiniProject-5

Classic Digit Classifier via Decision Tree

Student Name: Mustafa Sadeghi

Student ID: 4027390423

Delivery Deadline: 2025-05-17

Delivery Date: 2025-05-17

Project Abstract

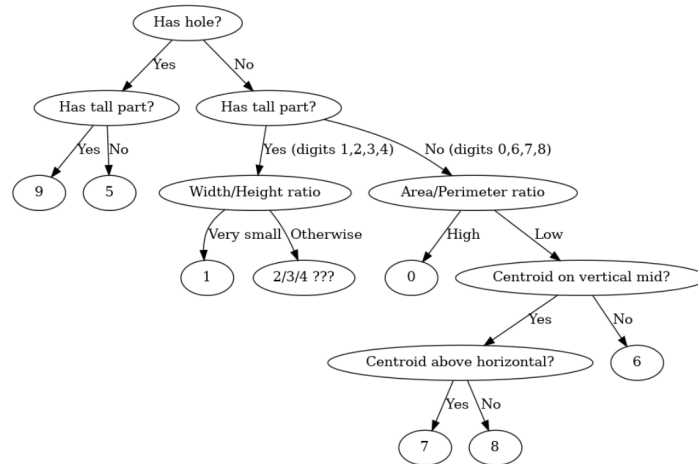


Figure 1: The decision tree provided in the project assignment.

In this MiniProject-5, I build a fully deterministic, rule-based decision-tree digit recognizer that combines classic image-processing steps—grayscale conversion, Otsu’s thresholding, contour extraction and height normalization—with a cascade of heuristic tests: loop (hole) detection for 5 and 9, polygonal approximation for perfect zeros, a “tall-stroke” node where 1 is isolated by a slim width, 4 by a pronounced right-lean curve, and 2 vs. 3 by counting morphological “teeth,” and finally centroid-based deviation for 6, 7, 8. All intermediate processing stages are systematically saved under the debug_steps folder, and the result is a transparent, interpretable pipeline that draws colored bounding boxes and labels on the original image.

Keywords: Centroid-Based Classification, Decision-tree classifier, Rule-based digit recognition.

1. Grayscale Conversion & Adaptive Thresholding

I convert the input image (“Im321.png”) to grayscale and binarize it using Otsu’s method, producing a clean binary inverse image for contour analysis.

2. Contour Extraction & Filtering

I use OpenCV’s `findContours` (with `RETR_EXTERNAL`) to detect all potential digit blobs, then discard any whose area falls below the `filter.area` threshold.

3. ROI Segmentation & Height Normalization

For each remaining contour, I extract its bounding box ROI and resize it—preserving aspect ratio—so its height matches `row_norm.row_height`, ensuring scale-invariant downstream tests.

4. Heuristic Decision-Tree Classification

- **Hole Detection Node (`_node_hole`):** retrieve second-level contours (`RETR_CCOMP`); if a hole exists, compute its centroid and label “9” for top-half holes or “5” otherwise.
- **Zero Approximation Node (`_node_zero`):** approximate the main contour as a polygon and check for 4 vertices, an aspect ratio near 1.0, and high solidity (> 0.9); label “0” if all pass.
- **Tall-Stroke Node (`_node_tall_stroke`):** apply a vertical morphological open; if present, then:
 - **Slim-Stroke Test:** $\text{aspect ratio} < \text{wh_ratio.slim} \rightarrow \text{“1”}$
 - **Right-Lean Curve Test:** pronounced curve on top band $\rightarrow \text{“4”}$
 - **Teeth-Counting Test:** accentuate teeth with dilations/erosions and count fragments; $\leq \text{two_three.teeth_thr} \rightarrow \text{“2,”}$ else $\rightarrow \text{“3.”}$
- **Centroid Node (`_node_centroid`):** compute the ROI’s intensity centroid; horizontal deviation $\rightarrow \text{“6,”}$ else vertical split at `centroid.y_frac` gives “7” (upper) or “8” (lower).

5.Debug Artifact Archival

I save every intermediate image in `debug_steps/overview/` (`threshold.png`, `all_contours.png`) and under `debug_steps/rois/roi_<n>/` (`raw.png`, `contour.png`, `norm.png`, `hole_mask.png`, `opened.png`, `top_band.png`, and `teeth.txt`). You can see the structure of `debug_steps/` folder below :

```
debug_steps/  
  overview/  
    threshold.png      # binary input  
    all_contours.png   # threshold + all detected contours  
  rois/  
    roi_0/  
      raw.png           # extracted ROI before normalization  
      contour.png       # raw ROI with main contour overlaid  
      norm.png          # height-normalized ROI  
      hole_mask.png     # binary mask of detected hole(s)  
      opened.png        # result of tall-stroke opening  
      top_band.png      # slice of the ROI's top fraction  
      teeth.txt         # integer count of "teeth"  
    roi_1/ ...
```

6.Final Annotation & Display

After labeling, I overlay colored bounding boxes and digit labels onto the original image and display only the final annotated result via Matplotlib.

The decision tree of the algorithm

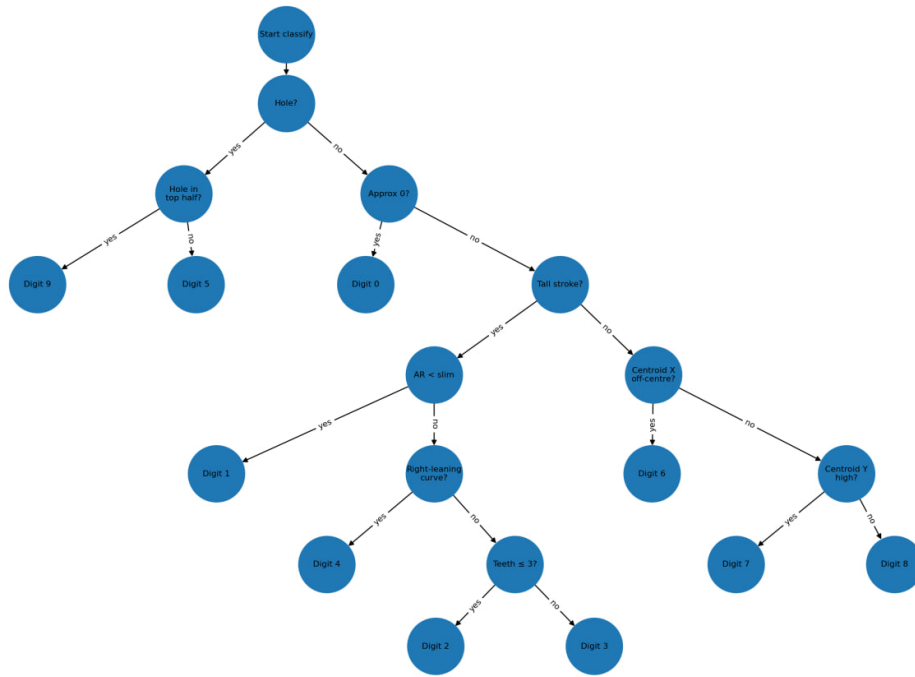


Figure 2: The decision tree of the digit classification algorithm

To fully distinguish 2, 3, and 4 I designed a small subtree of purely morphological and geometric tests: after confirming a tall stroke, I first isolate a narrow top band and fit its largest contour with a minimum-area rectangle—if that rectangle’s aspect ratio and orientation exceed calibrated thresholds, I label it as a right-leaning “4.” For any remaining ROIs, I then apply a sequence of dilations and erosions that accentuate the small “teeth” protrusions characteristic of “2,” subtract the final eroded image to isolate those fragments, and count them—if the count falls at or below my threshold it becomes “2,” otherwise it is classified as “3.” This addition completes the instructor’s framework with a clear, deterministic split among 2, 3, and 4.

Final Result

The annotated result shows each Persian digit neatly boxed and labeled from 9 down to 0. Despite variations in stroke width and orientation, the rule-based decision tree—using hole detection, polygon fitting, morphological stroke analysis, curve tests, teeth counting, and centroid checks—consistently isolates and classifies every digit.



Figure 3: The final result of the algorithm.

Table 1: Table of the algorithm’s parameters.

Parameter	Value	Purpose	Stage
wh_ratio.tall	0.59	Threshold for detecting a tall vertical stroke	Tall-Stroke Node
wh_ratio.slim	0.35	Max width/height ratio for classifying “1”	Tall-Stroke Node
centroid.x_deviation	0.03	Horizontal offset fraction for detecting “6”	Centroid Node
centroid.y_frac	0.48	Vertical split fraction for “7” vs. “8”	Centroid Node
filter.area	40	Min contour area to retain as digit candidate	Contour Extraction
row_norm.row_height	50	Height (px) to which all ROIs are resized	ROI Segmentation
row_norm.pad	1	Padding (px) around each ROI when cropping	ROI Segmentation
zero.eps	0.03	Epsilon for polygonal approximation in zero test	Zero Node
zero.vertices	4	Required vertex count to detect “0”	Zero Node
zero.ar_low	0.8	Min aspect ratio for a valid “0” shape	Zero Node
zero.ar_high	1.2	Max aspect ratio for a valid “0” shape	Zero Node
zero.solidity_thr	0.9	Solidity threshold for confirming a closed loop	Zero Node
recon.se_frac	0.6	SE height fraction for morphological opening	Tall-Stroke Node
recon.area_frac_thr	0.31	Min opened-area ratio to detect tall stroke	Tall-Stroke Node
persian.top_frac	0.1	Top-band height fraction for curve analysis	Right-Lean Curve Node
persian.curve_aspect	3	Aspect-ratio threshold for right-lean curve	Right-Lean Curve Node
ensemble.right_curve_angle_thr	50	Angle threshold for detecting right-lean curve	Right-Lean Curve Node
two_three.teeth_thr	3	Max “teeth” count to distinguish “2” from “3”	Teeth-Counting Test